

# *Mastering Helm Charts*

A comprehensive guide to Kubernetes package management



# Agenda

## ***Introduction to Helm***

Understanding the Kubernetes package manager

## ***Helm Architecture***

Components and how they work together

## ***Helm Chart Structure***

Anatomy of charts and templates

## ***Hands-on with Helm Charts***

Practical commands and implementations

## ***Advanced Helm Features***

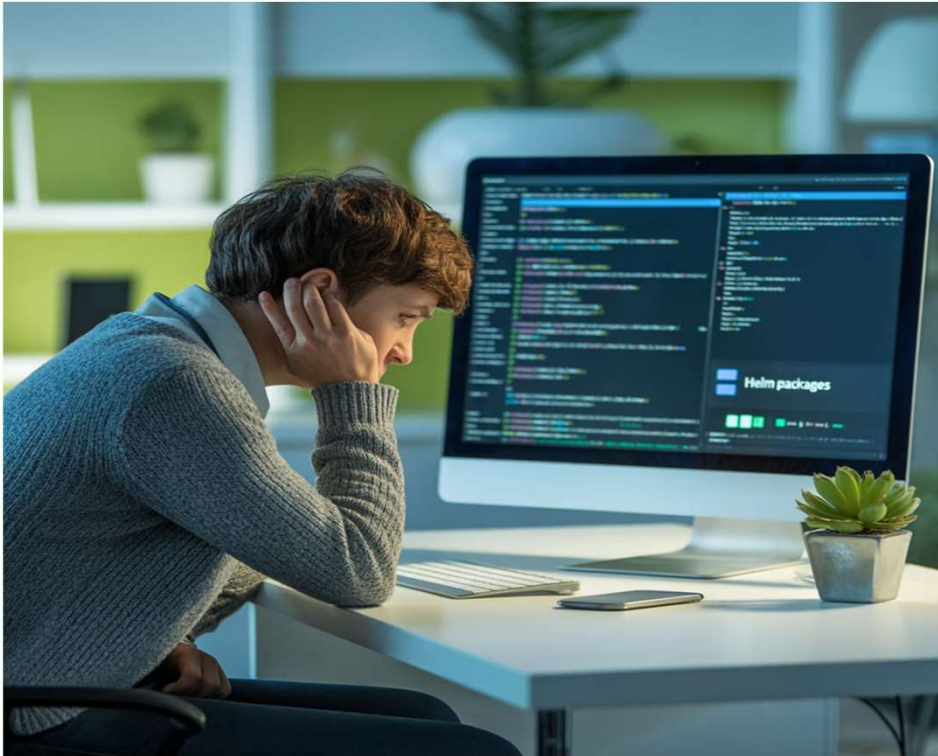
Hooks, dependencies, and conditionals

## ***Best Practices & Use Cases***

Real-world applications and recommendations

## *Why Helm?*

Managing Kubernetes applications involves multiple YAML files that can be difficult to track and maintain. Helm solves this by acting as a package manager for Kubernetes.



### ***Key Benefits***

Package, share, and reuse applications

Version-controlled deployments

Easy upgrades and rollbacks

# *Helm vs kubectl*

## *kubectl*

Applies static YAML files directly to the Kubernetes cluster

❌ Manual, error-prone process

```
kubectl apply -f deployment.yaml  
kubectl apply -f service.yaml  
kubectl apply -f ingress.yaml
```

## *Helm*

Applies templates with configurable values

✅ Dynamic, configurable, repeatable

```
helm install myapp ./myapp
```

Helm simplifies deployment by packaging multiple resources together with configurable parameters.

## Demo: Installing from a Repository

Let's see how easy it is to install an application using Helm:

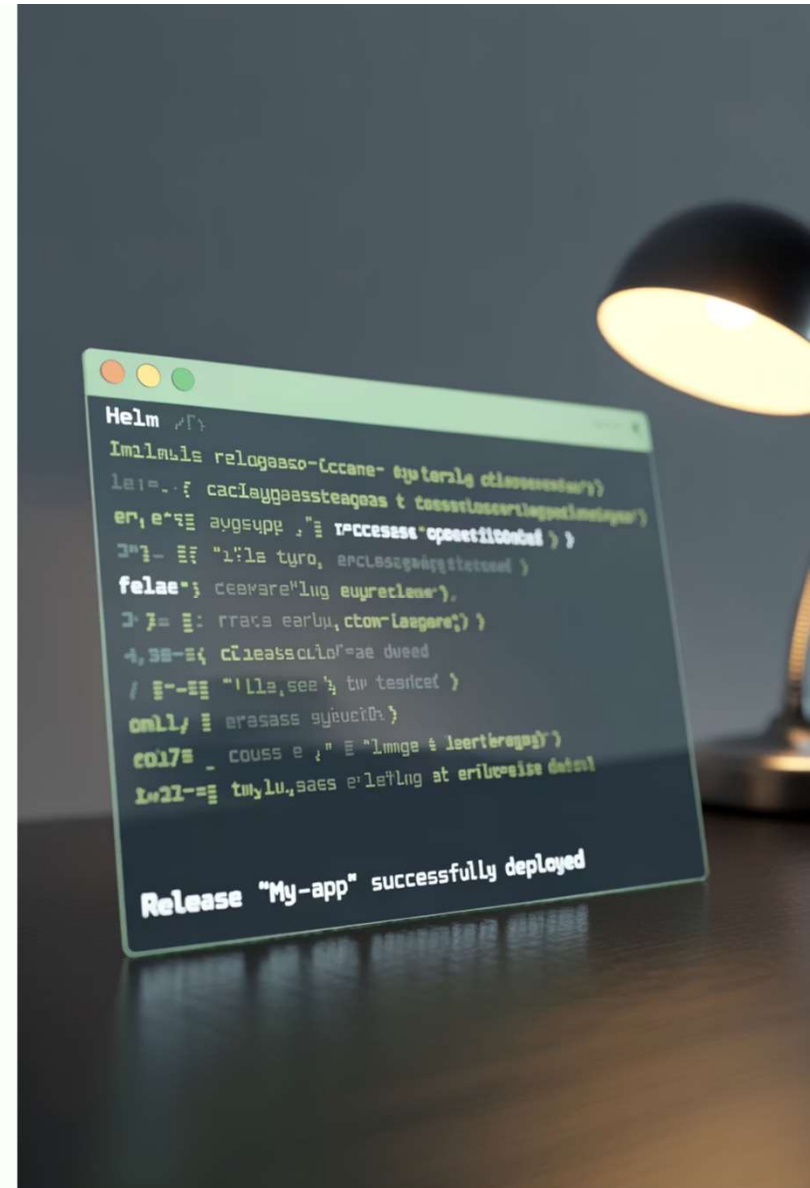
```
helm repo add bitnami https://charts.bitnami.com/bitnami  
helm search repo nginx  
helm install my-nginx bitnami/nginx
```

With just three commands, we've added a repository, searched for a chart, and deployed an application!

? Which of the following is NOT a benefit of Helm?

- A) Simplifies app deployment
- B) Automates chart upgrades and rollbacks
- C) Provides built-in monitoring for pods
- D) Reuses configuration templates

Answer: C



# *Helm Components*



## *Helm CLI*

The command-line interface that interacts with Kubernetes API and manages charts and releases



## *Repository*

Storage location for charts, similar to Docker Hub for container images



## *Charts*

Packages of pre-configured Kubernetes resources, like a bundle of YAML files



## *Release*

An instance of a chart running in a Kubernetes cluster with specific configuration

# *Helm Architecture*



Helm simplifies the deployment process by handling the complexity of managing multiple Kubernetes resources through a single interface.

# Release Lifecycle



## *helm install*

Creates a new release from a chart



## *helm upgrade*

Updates an existing release with new values



## *helm rollback*

Restores a previous release version



## *helm uninstall*

Removes a release from the cluster

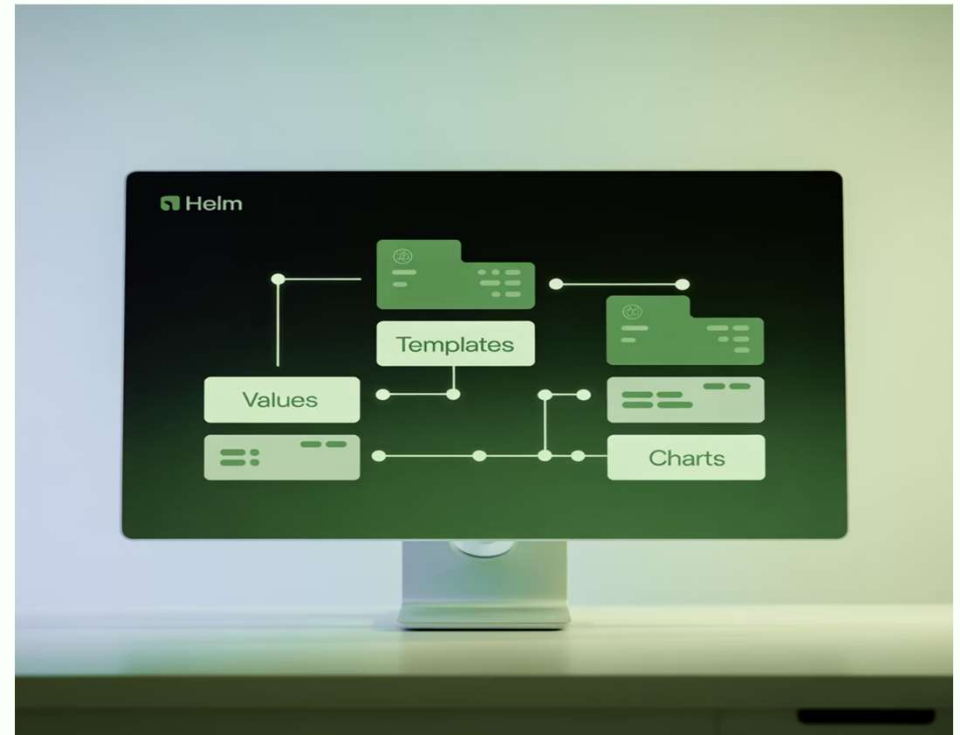
Helm tracks each deployment as a release, making it easy to manage the entire lifecycle of your applications.



## Helm Chart Structure

```
mychart/  Chart.yaml      # Chart metadata
values.yaml  # Default values
templates/  # K8s manifest templates
charts/     # Dependencies
NOTES.txt   # Post-install notes
```

A Helm chart is organized in a specific directory structure that contains all the necessary files to deploy an application.



Each component serves a specific purpose in the chart's functionality, from defining metadata to providing templates for Kubernetes resources.

# *Chart.yaml (Metadata)*

The Chart.yaml file contains essential metadata about your Helm chart:

```
apiVersion: v2
name: myapp
description: A simple Helm chart for Kubernetes
version: 0.1.0
appVersion: "1.0"
```

## ***apiVersion***

Specifies the Helm API version (v2 is current)

## ***name***

The name of the chart

## ***version***

The chart version (follows SemVer)

## ***appVersion***

The version of the application being deployed

## *values.yaml (Defaults)*

The values.yaml file defines default configuration values that can be overridden during installation:

```
replicaCount: 2
image: repository: nginx
tag: "1.25"service:
type: ClusterIP
port: 80
```

These values can be referenced in templates using the `{{ .Values }}` object, making charts highly configurable without modifying the templates themselves.



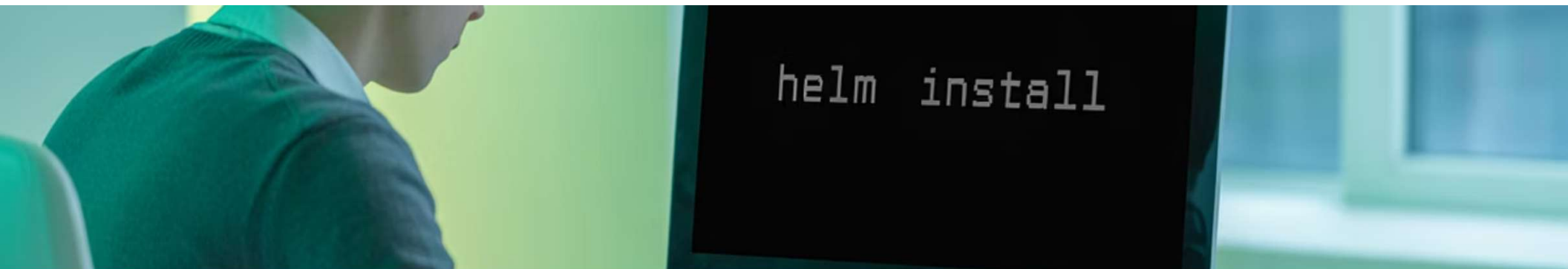
## *templates/deployment.yaml*

Templates use Go templating syntax to insert values dynamically:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-{{ .Chart.Name }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Chart.Name }}
  template:
    metadata:
      labels:
        app: {{ .Chart.Name }}
    spec:
      ② containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{
.Values.image.tag }}"
          ports:
            - containerPort: {{ .Values.service.port }}
```

You need to deploy an app with 5 replicas and NodePort service. Which file should you edit?

- A) Chart.yaml
- B) values.yaml
- C) deployment.yaml directly
- D) NOTES.txt



## *Basic Helm Commands*

01

### *Create a new chart*

```
helm create myapp
```

Generates a new chart with default structure and templates

02

### *Install a chart*

```
helm install myapp ./myapp
```

Deploys the chart to your Kubernetes cluster

03

### *List releases*

```
helm list
```

Shows all deployed releases in the current namespace

# *Overriding Values*

## *Using --set flag*

```
helm install myapp ./myapp \ --set replicaCount=3
```

Overrides individual values directly from the command line

## *Using custom values file*

```
helm install myapp ./myapp \ -f custom-values.yaml
```

Overrides values using a separate YAML file with custom configurations

These methods allow you to customize deployments without modifying the original chart files, making it easy to maintain different configurations for different environments.

# *Upgrade and Rollback*

## *Upgrade a release*

1

```
helm upgrade myapp ./myapp --set replicaCount=4
```

Updates an existing release with new values or chart version

## *Rollback to previous version*

3

```
helm rollback myapp 1
```

Reverts to a specific revision number (in this case, revision 1)

2

## *View release history*

```
helm history myapp
```

Shows all revisions of a release for tracking changes

Helm's version control capabilities make it easy to manage application lifecycle and recover from problematic deployments.

# *Debugging Helm Charts*

## *Preview rendered templates*

```
helm template ./myapp
```

Shows the YAML that would be applied without actually installing

## *Inspect a release*

```
helm get all myapp
```

Displays all information about a release

## *Uninstall a release*

```
helm uninstall myapp
```

Removes a release from the cluster

## *Practice Lab Task*

1. Create a chart named demoapp
2. Deploy with 2 replicas
3. Upgrade to 4 replicas
4. Rollback to previous version





# *Helm Hooks*

Hooks allow you to intervene at specific points in a release's lifecycle:

```
apiVersion: batch/v1kind: Jobmetadata:  name: "{{ .Release.Name }}-db-init"  annotations:    "helm.sh/hook": pre-installspec:  template:    spec:      containers:        - name: db-init          image: postgres:15
```

## *Common Hook Types*

- pre-install: Run before the release is installed
- post-install: Run after the release is installed
- pre-delete: Run before a release is deleted

## *Use Cases*

- Database migrations
- Secret setup
- Resource cleanup

# Chart Dependencies

Charts can depend on other charts, creating a modular architecture:

```
dependencies:- name: redis  version: "16.13.2"  repository: "https://charts.bitnami.com/bitnami"
```

Define dependencies in `Chart.yaml` and store sub-charts inside the `charts/` directory.

## Managing Dependencies

```
# Update dependencieshelm dependency update ./myapp# Build dependencieshelm dependency build ./myapp
```



# Conditionals and Loops

## Conditionals

```
{{- if .Values.service.enabled }}apiVersion: v1kind:
Servicemetadata:  name: {{ .Release.Name }}-service{{- end }}
```

Only include resources when certain conditions are met

## Loops

```
{{- range $name, $port := .Values.service.ports }}- name: {{
$name }}  port: {{ $port }}{{- end }}
```

Iterate over collections to generate repeated blocks

? Which Helm feature is best for running a database initialization job before installing a chart?

- A) Conditionals
- B) Loops
- C) Hooks
- D) Dependencies

Answer: C



# *Best Practices & Real-world Use Cases*

## *Best Practices*

- Keep values.yaml clean and minimal
- Use CI/CD pipelines for Helm deployments
- Use helm lint for validation
- Version lock dependencies

## *Real-world Use Cases*

- Deploy NGINX Ingress Controller
- Deploy Prometheus + Grafana stack
- Deploy custom microservices

## *Key Takeaways*

Helm simplifies Kubernetes app management

Charts = reusable, parameterized packages

Releases = tracked versions of charts

Advanced features → hooks, dependencies, conditionals